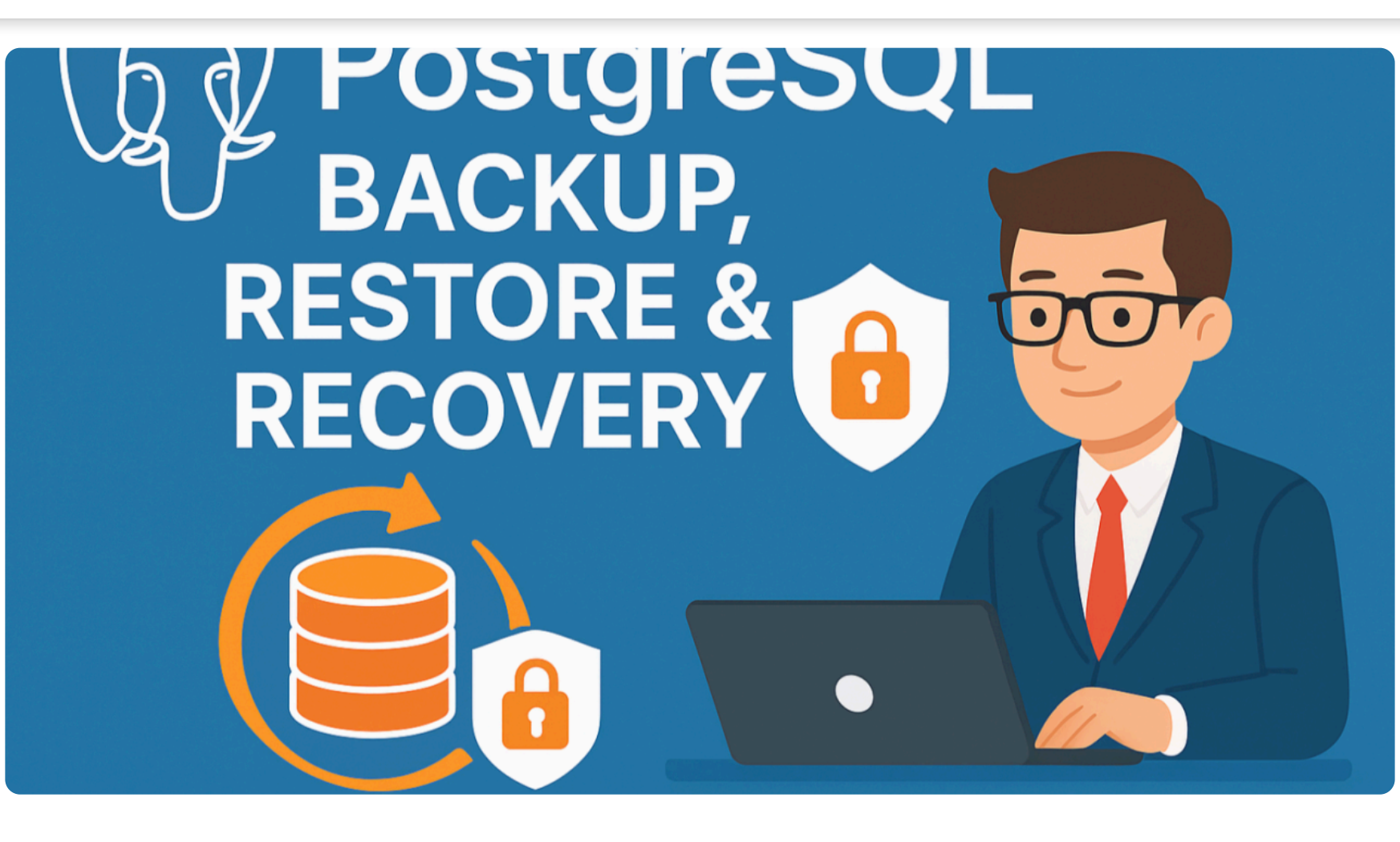

PostgreSQL – Backup, Restore & Recovery — Foundations Series

Wayne C.
Published Nov 6, 2025

+ Follow

Physical & Logical Backups, PITR, Incremental Backups (v17), and Maintenance Best Practices

In the world of enterprise databases, **data durability** isn't a luxury—it's survival. PostgreSQL, known for its robustness, provides flexible and powerful tools to manage **backups, restores, and recovery**. Whether you're a solo developer or running a multi-

Like Comment Share

compliance, and peace of mind.

🔐 1. Backup Types in PostgreSQL

PostgreSQL supports **two primary forms of backups**:

🔶 **Logical Backups**

These are higher-level backups using PostgreSQL tools to extract schema and data.

Tools:

- pg_dump
- pg_dumpall
- pg_restore

🔶 **Physical Backups**

These are binary copies of the PostgreSQL data directory and WAL (Write-Ahead Log) files.

Tools:

- pg_basebackup
- pgBackRest (third-party, enterprise-grade)

🧰 2. Logical Backup and Restore
✅ **pg_dump**

Creates a consistent dump of a database (schema + data).

```
pg_dump -U myuser -d mydb -F c -f mydb.dump
```

✅ **pg_dumpall**

Backs up the entire PostgreSQL cluster, including globals (roles, tablespaces).

```
pg_dumpall -U postgres > full_cluster.sql
```

✅ **pg_restore**

Restores from a custom-format dump.

```
pg_restore -U myuser -d newdb mydb.dump
```

Use Cases:

- Portability
- Upgrades
- Single-database recovery

🗃 3. Physical Backup and Restore
✅ **pg_basebackup**

Creates a binary copy of the database for full cluster restoration.

```
pg_basebackup -D /data/backup -Fp -Xs -P -U replicator
```

Restoring:

1. Stop PostgreSQL.
2. Replace the data directory with backup.
3. Start PostgreSQL.

Note: Ensure WAL files are intact or available.

⏳ 4. Point-in-Time Recovery (PITR)
What Is It?

Allows restoration to any point before a disaster (within the WAL retention window).

Steps:

1. Take a physical backup.
2. Enable WAL archiving.
3. Set recovery target in recovery.conf or postgresql.auto.conf.

```
restore_command = 'cp /path/to/archive/%f %p'
recovery_target_time = '2025-11-06 03:15:00'
```

PostgreSQL replays WAL logs until the recovery target is reached.

Use Cases:

- Human error correction
- Corruption rollback
- Auditing & forensics

🔄 5. Incremental Backups (PostgreSQL 17+)

As of PostgreSQL 17, native support for **incremental backups** (via **page-level change tracking**) is introduced.

Benefits:

- Faster backups
- Reduced I/O
- Cloud-optimized

Example Tooling (third-party):

- pgBackRest with --type=incremental
- Native PostgreSQL 17 features (with track_io_timing)

```
pg_basebackup --incremental --target-dir=/backup/inc_01
```

🔍 6. Recovery Scenarios
A. Full Cluster Restore

- Use physical backup and WAL
- Recreate data directory
- Reapply WAL for consistency

B. Single Database Restore

- Use pg_dump + pg_restore
- Create new DB and restore

C. PITR to Specific Time

- Use recovery_target_time in config

D. Table-Level Recovery

- Use pg_dump to restore only required table

🛠 7. Maintenance Activities
✅ **Vacuum**

Reclaims storage from dead tuples.

```
VACUUM;
```

✅ **Analyze**

Updates statistics for query planner.

```
ANALYZE;
```

✅ **Reindex**

Rebuilds corrupted or bloated indexes.

```
REINDEX DATABASE mydb;
```

✅ **Check Logs & Disk Usage**

Ensure WAL archive is not full and logs are rotated.

```
du -sh /var/lib/postgresql/data/pg_wal
```

☁️ 8. Cloud Backup Strategies
AWS RDS

- **Automated Snapshots**
- **Manual pg_dump to S3**

Snapshots

🔧 **How to Configure Automated Snapshots**
A. While creating the RDS instance:

1. Use the AWS Console or CLI
2. Under **Backup**, configure: a) **Backup retention period** (e.g., 7 days) b) **Backup window** (when daily backups should occur) c) Enable **Enable automated backups**

B. Modify an existing instance:

```
aws rds modify-db-instance \
  --db-instance-identifier mypostgresdb \
  --backup-retention-period 7 \
  --preferred-backup-window 02:00-03:00 \
  --apply-immediately
```

Viewing Automated Snapshots

Use the AWS Console:

- Go to **RDS > Snapshots**
- Filter by **Automated Snapshots**

Or use AWS CLI:

```
aws rds describe-db-snapshots \
  --db-instance-identifier mypostgresdb \
  --snapshot-type automated
```

Restoring from an Automated Snapshot

```
aws rds restore-db-instance-from-db-snapshot \
  --db-instance-identifier mypostgresdb-restore \
  --db-snapshot-identifier rds:my-postgres-snap-2025-11-06-02-00
```

Notes

- Automated backups are **automatically deleted** when the instance is deleted unless **final snapshot** is requested.
- PITR (Point-in-Time Recovery) is available using transaction logs + snapshots.
- Snapshots are stored in **S3** but **not directly accessible** — you restore them via RDS, not aws s3 cp.

AWS RDS - pg_dump to S3

```
pg_dump | aws s3 cp - s3://my-bucket/daily.sql
```

GCP Cloud SQL

- Scheduled backups via console
- Export to GCS using SQL or gcloud

📦 **Option 1: Export via gcloud CLI**

```
gcloud sql export sql INSTANCE_NAME \
  gs://YOUR_BUCKET_NAME/your_export.sql.gz \
  --database=your_database_name \
  --offload
```

Example:

```
gcloud sql export sql my-postgres-instance \
  gs://my-pg-backups/daily/devdb_2025_11_06.sql.gz \
  --database=devdb \
  --offload
```

- INSTANCE_NAME: your Cloud SQL instance
- --offload: optional flag to reduce impact on the DB instance
- Result: compressed .sql file in GCS bucket

📘 **Option 2: Use GCP Console**

1. Go to your **Cloud SQL** instance.
2. Navigate to **Exports** → Click **EXPORT**.
3. Select: **Database -> Export format**: SQL or CSV -> **Destination**: GCS bucket path

Azure Database for PostgreSQL

- Built-in geo-redundant backups
- Use Blob storage for logical backups

✅ **1. Built-In Geo-Redundant Backups**

Azure Database for PostgreSQL (both **Single Server** and **Flexible Server**) supports **automatic backups**, and you can configure them for **geo-redundancy**:

🔒 **Features:**

- **Automatic full backups**, transaction logs, and daily snapshots
- **Retention**: 7–35 days (configurable)
- **Geo-Redundant**: Stores backups in a paired region for disaster recovery

🔧 **How to Enable:**

1. Go to **Azure Portal**
2. Navigate to your PostgreSQL server
3. Under **Backups**: a) Set **Backup Redundancy Option** to Geo-Redundant b) Choose **Backup Retention Period** (7–35 days)

📦 **Step-by-Step: Export with pg_dump to Blob**
A. Install Azure CLI and authenticate:

```
az login
```

B. Create a storage container (if needed):

```
az storage account create \
  --name mypgstorage \
  --resource-group myresourcegroup \
  --location eastus \
  --sku Standard_LRS

az storage container create \
  --account-name mypgstorage \
  --name pgbackups \
  --auth-mode login
```

C. Run the logical backup and upload:

```
# Backup
pg_dump -h your-db.postgres.database.azure.com \
  -U dbuser@your-db -d yourdatabase \
  -F c -f devdb_$(date +%F).dump

# Upload to Blob
az storage blob upload \
  --account-name mypgstorage \
  --container-name pgbackups \
  --name devdb_$(date +%F).dump \
  --file devdb_$(date +%F).dump \
  --auth-mode login
```

📅 9. Scheduling Backups

Use cron, systemd timers, or pg_cron to automate:

```
0 2 * * * pg_dump -U postgres mydb > /backups/mydb_$(date +\%F).sql
```

⚙️ 10. Monitoring and Verification

- Test restores weekly
- Use checksum tools
- Monitor backup completion, failure, size deltas

🧱 11. Best Practices

- Always test restores.
- Use a combination of logical + physical backups.
- Store backups **offsite** (cloud, remote FS).
- Retain at least 7–30 days of WAL.
- Automate and monitor everything.

✅ Conclusion

PostgreSQL offers **battle-tested backup and recovery tools**, giving DBAs and DevOps engineers deep control over disaster recovery and maintenance. Whether it's a dropped table, a regional outage, or accidental data corruption, PostgreSQL's backup ecosystem ensures you can **bounce back fast**—with confidence.

To view or add a comment, sign in

More articles by Wayne C.

PostgreSQL
Advanced Features

Nov 5, 2025
PostgreSQL – Advanced Features for Modern Data...
Joins • Partitioning • Inheritance • MVCC • Constraints Management PostgreSQL isn't just an open-source relational...
1

PostgreSQL
aws
RDS

Nov 5, 2025
PostgreSQL on AWS RDS: Architecture, Management, and...
Instance Creation • Upgrades • PITR • Monitoring • Parameter Groups • Security Groups Running PostgreSQL on Amazon...

Nov 5, 2025
PostgreSQL: Monitoring & Performance Tuning — PostgreSQ...
Designing for Speed, Stability, and Scalability Performance tuning in PostgreSQL isn't a single switch you flip — it's...

Nov 4, 2025
PostgreSQL: Extensions That Power Insight, Performance, a...
PostgreSQL is widely respected for its reliability and standards compliance — but its true power lies in extensibility....
2

Show more

Explore content categories

Career Productivity Finance Soft Skills & Emotional Intelligence Project Management Education

Show more